

# AI Operations (AIOps)

## Module 3: Infrastructure & Containerization

Course Content — 2 Lectures (6 Contact Hours) | Third-Year B.Tech. Data Science

*REVISION NOTE: Lecture 2 (Kubernetes & KServe) has been rewritten around a CPU-only, hands-on-verified workload so every student can run it on a laptop, with no GPU required. Lecture 1 (Docker) is unchanged.*

### Module Overview

Module 1 made a single model reproducible. Module 2 made data and compute scale across a cluster. Module 3 answers the question both of those modules quietly depend on: what actually IS “a machine” in a distributed AI system, and who decides which workload runs where? This module covers the infrastructure layer beneath every previous lecture — Docker for packaging reproducible, portable environments, and Kubernetes for orchestrating those containers across a cluster of CPUs and nodes, culminating in a hands-on demonstration of real horizontal scaling that every student runs and measures themselves.

| Lecture | Title                                                              | Duration | Core Focus                                                                                                                          |
|---------|--------------------------------------------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------|
| L1      | Docker & Containerization for AI                                   | ~3.0 hrs | Images vs. containers; Dockerfiles; multi-stage builds; GPU passthrough                                                             |
| L2      | Kubernetes for AI/ML Workloads & Scaling Model Serving with KServe | ~3.0 hrs | minikube setup; CPU-only distributed training; single-node & multi-node scaling; scalability testing; HPA-based serving autoscaling |

### Module Learning Outcomes

- Explain why containerization is the natural infrastructure answer to Module 1's reproducibility protocol (CO2, CO9).
- Write a production-quality, multi-stage Dockerfile for an AI application, minimizing image size and build time (CO2).
- Configure GPU passthrough so a containerized workload can access host GPU resources, conceptually and via the exact commands required (CO2, CO3).
- Stand up a local single-node and multi-node Kubernetes cluster with minikube (CO2, CO3).
- Design and run a CPU-only, embarrassingly parallel distributed workload as a Kubernetes Indexed Job, scaling it across multiple pods on one node and then across multiple nodes (CO3, CO4).
- Empirically measure how wall-clock completion time scales with Job parallelism, and explain the sources of sub-linear speedup (CO4, CO6).
- Deploy a CPU-only model-serving application behind a HorizontalPodAutoscaler, load-test it, and observe replica scaling across multiple nodes (CO3, CO4).
- Explain what a KServe InferenceService automates relative to a hand-written Deployment/Service/HPA trio, and the production tradeoffs of RawDeployment vs. Knative-based serverless modes (CO3, CO4).



# Lecture 1 — Docker & Containerization for AI (≈3 hours)

## 1.1 Why Containers, After Everything We've Already Built?

Module 1 built a reproducibility protocol: Git for code, Conda/Mamba for the environment, DVC for data, MLflow for the run. That protocol has one weak link: a Conda environment file describes Python packages, but not the operating system, system libraries, CUDA/cuDNN versions, or anything outside the Python process itself. Two machines with an IDENTICAL environment.yml can still behave differently if their underlying OS or GPU drivers differ. Containers solve this by packaging the ENTIRE runtime environment — OS libraries included — into a single, portable unit.

## 1.2 Images vs. Containers

- **Image** — a read-only, layered template: a filesystem snapshot plus metadata (what command to run, what ports to expose). Built once from a Dockerfile.
- **Container** — a running (or stopped) INSTANCE of an image, with its own writable layer on top. You can run many containers from one image.
- **Registry** — a place images are stored and shared (Docker Hub, a private registry, a cloud provider's registry).

## 1.3 Anatomy of a Dockerfile

```
FROM python:3.10-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["python", "train.py"]
```

Docker caches each layer and reuses it on the next build if nothing that layer depends on has changed. This is why requirements.txt is copied and installed BEFORE the rest of the application code.

## 1.4 Multi-Stage Builds

```
# --- Stage 1: build ---
FROM python:3.10 AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --target=/build/deps -r requirements.txt

# --- Stage 2: minimal runtime ---
FROM python:3.10-slim
WORKDIR /app
COPY --from=builder /build/deps /usr/local/lib/python3.10/site-packages
COPY . .
CMD ["python", "serve.py"]
```

The final image contains only the Python runtime and installed packages — no compilers, no build caches, no dev headers. This directly reduces image size, pull time, attack surface, and cold-start latency — a theme that returns in Lecture 2's serving-scalability discussion.

## 1.5 GPU Passthrough

```
$ docker run --gpus all my-training-image  
$ docker run --gpus '"device=0"' my-training-image
```

Requires the NVIDIA Container Toolkit on the host. The CUDA version inside the container's base image must be COMPATIBLE with the host's installed NVIDIA driver, not identical to it.

## 1.6 Case Study: Optimizing GPU Access on a Shared University Lab Machine

Scenario: a single university lab workstation has 2 GPUs and is shared by several student research groups. Both groups' containers pass `--gpus all`, so both contend for the SAME GPUs, causing out-of-memory errors for both. The container-level fix is explicit device assignment (`--gpus '"device=0"'` vs. `'"device=1"'`); the limitation is that this doesn't scale past a handful of manually-coordinated users — which motivates Lecture 2's cluster-level scheduling.

## 1.7 Hands-on Lab: Building a Multi-Stage, GPU-Ready Docker Image

Every student builds a naive single-stage image and a multi-stage image for the same training script, documents the size reduction, adds a `.dockerignore`, and reasons through the exact `--gpus` flag syntax for both machines with and without GPU hardware.

## Lecture 2 — Kubernetes for AI/ML Workloads & Scaling Model Serving with KServe (≈3 hours)

### Part A: Kubernetes for AI/ML Workloads (≈100 minutes)

#### 2.1 From One Container to a Fleet

Docker answers “how do I package one reproducible unit of work?” Kubernetes answers the next question: “I have dozens of these containers and several machines — who decides what runs where, restarts what fails, and shares resources fairly?”

#### 2.2 Core Kubernetes Objects

| Object     | What It Is                                                                                    | AI/ML Relevance                                       |
|------------|-----------------------------------------------------------------------------------------------|-------------------------------------------------------|
| Pod        | The smallest deployable unit — one or more tightly-coupled containers sharing storage/network | One training or inference container usually = one Pod |
| Deployment | Manages a set of identical Pod replicas, handles rolling updates                              | Scaling an inference service horizontally (Part B)    |
| Job        | Runs Pods to COMPLETION (not forever) — tracks success/failure, can retry                     | Today's hands-on: a distributed hyperparameter search |
| Service    | A stable network endpoint routing traffic to a changing set of Pods                           | Exposing an inference endpoint at a fixed address     |
| Node       | A physical or virtual machine in the cluster that runs Pods                                   | One minikube node per “machine” in today's cluster    |

#### 2.3 Jobs vs. Deployments: Training vs. Serving

A TRAINING run is naturally a Kubernetes Job: run to completion, potentially retried on failure, then done. An INFERENCE service is naturally a Deployment: run forever, serving traffic, scaled up/down with demand. Deploying a training script as a Deployment causes Kubernetes to treat its successful exit as a failure to maintain the desired replica count, restarting it indefinitely.

#### 2.4 Infrastructure as Code (IaC) Principles

Every Kubernetes object in this lecture is defined in a declarative YAML manifest, not created via ad hoc commands — the same reproducibility principle from Module 1, now applied to infrastructure itself.

## 2.5 Setting Up minikube: Single-Node, Then Multi-Node

Every student installs minikube and kubectl, starts a single 4-CPU node, and later scales the SAME cluster (or starts fresh) with three nodes. minikube nodes run as containers on the student's own machine — not separate physical hardware — but Kubernetes schedules across them exactly as it would across a real multi-machine cluster, which is what makes this a faithful teaching substitute for a production cluster.

```
# Install (macOS)
brew install minikube kubectl

# Install (Linux)
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Start a single-node, 4-CPU cluster
$ minikube start --cpus 4 --memory 4096 --driver=docker
$ kubectl get nodes -o wide

# Later: scale to 3 nodes
$ minikube node add
$ minikube node add
$ kubectl get nodes -o wide
```

## 2.6 The Workload: Distributed Hyperparameter Search

This module's hands-on workload is a distributed hyperparameter search: training a RandomForestClassifier on a small (6,000-row, 20-feature), deterministic synthetic dataset, once for each of 12 hyperparameter combinations (4 values of `n_estimators` × 3 values of `max_depth`). Each run is fully independent — an EMBARRASSINGLY PARALLEL workload, exactly the shape Kubernetes Jobs are built for. It requires no GPU and each run finishes in well under three seconds, so the whole exercise is comfortable on a laptop.

### Why this workload, specifically

A hyperparameter search lets us demonstrate real, measurable Kubernetes-level parallelism without any GPU, without any inter-pod communication or networking library, and without any dataset larger than a couple of megabytes — removing every source of classroom fragility while keeping the underlying lesson (Kubernetes schedules independent units of work across CPUs and nodes) fully intact.

## 2.7 Indexed Jobs: completions, parallelism, completionMode

| Field                   | Meaning                                                                                                                      |
|-------------------------|------------------------------------------------------------------------------------------------------------------------------|
| completions             | Total number of pods that must succeed — our fixed grid size (12)                                                            |
| parallelism             | How many pods run CONCURRENTLY at once — what we vary to test scaling                                                        |
| completionMode: Indexed | Kubernetes assigns each pod a unique index (0..completions-1) via the <code>JOB_COMPLETION_INDEX</code> environment variable |

```
import os
import itertools

N_ESTIMATORS_GRID = [50, 100, 150, 200]
```

```
MAX_DEPTH_GRID = [4, 8, 12]
GRID = list(itertools.product(N_ESTIMATORS_GRID, MAX_DEPTH_GRID)) # 12 combinations

completion_index = int(os.environ.get("JOB_COMPLETION_INDEX", "0"))
n_estimators, max_depth = GRID[completion_index % len(GRID)]
```

Each worker deliberately sets `n_jobs=1` on its `RandomForestClassifier`: this ensures one pod uses roughly one CPU core, so the parallelism story is about KUBERNETES scheduling many pods, not about a single model internally fanning out across cores. A pod's own name and node are populated via the Kubernetes Downward API (`fieldRef: metadata.name / spec.nodeName`), which is what lets the class later prove, after the fact, exactly which node ran which piece of work.

## 2.8 Hands-on: Single-Node, Multi-Pod

The worker script is containerized with a multi-stage Dockerfile (Lecture 1 pattern) with the dataset baked directly into the image, and built directly into minikube's image store with `minikube image build` — no external registry needed. The Job manifest sets `completions: 12` and `parallelism: 4`, so up to 4 pods run concurrently on the single available node. Running `kubectl get pods -o wide -w` live shows up to 4 pods Running simultaneously, all on the same node, with the remaining 8 combinations queuing and running as earlier ones finish.

## 2.9 Collecting Results: Why Not a Shared Volume

The obvious approach — have every pod write its result to a shared `PersistentVolume` — breaks in the multi-node exercise: minikube's default storage provisioner binds a volume to ONE node, so a pod scheduled on a different node cannot reliably read or write it. Instead, each pod prints a single machine-readable result line to `stdout`, and a collection script uses the official Kubernetes Python client to list every Pod belonging to the Job (via its auto-applied `job-name` label), fetch each Pod's logs, and parse out the result — a mechanism that works identically regardless of which node a Pod ran on.

## 2.10 Hands-on: Scaling to a 3-Node Cluster

After adding two more nodes with `minikube node add`, the SAME worker image must be explicitly loaded onto every node (`minikube image load hp-search-worker:latest`) — an image built or loaded on one node is not automatically present on the others, and a pod scheduled on a node without the image will show `ImagePullBackOff`. The multi-node Job manifest raises `parallelism` to 6 and adds a `topologySpreadConstraints` block that actively balances pods across nodes rather than incidentally packing them onto whichever node the scheduler checks first. Watching `kubectl get pods -o wide -w` now shows the `NODE` column varying across all three nodes — direct, visual proof that Kubernetes is scheduling work across multiple machines, not just multiple processes on one.

## 2.11 Scalability Testing: Does More Parallelism Actually Help?

A dedicated script resubmits the SAME fixed total work (`completions=12`) at increasing parallelism values (1, 2, 4, and optionally 6 on the 3-node cluster), measuring wall-clock completion time for each via the Kubernetes API, and plots both the raw time and the resulting speedup relative to `parallelism=1`.

### Reference results (a typical run)

`parallelism=1` → ~14.2s; `parallelism=2` → ~7.6s (~1.9x speedup); `parallelism=4` → ~4.3s (~3.3x speedup). The sub-linear speedup at `parallelism=4` is itself a valuable discussion point: with only 4 concurrent slots and 12 total jobs of uneven size (0.3s to 2.8s each), the LAST-finishing pod in the final batch sets the observed wall-clock time, and scheduling/startup overhead grows with more concurrent pods.

## 2.12 GPU Scheduling (Conceptual)

```
resources:  
requests:  
  nvidia.com/gpu: 1  
limits:  
  nvidia.com/gpu: 1 # GPUs cannot be overcommitted -- request must equal limit
```

GPU resources follow exactly the same request/limit pattern used throughout today's hands-on with `cpu:`, just with a different resource name, advertised to the scheduler by a device plugin (typically the NVIDIA device plugin). The one hard difference: unlike CPU, GPU requests and limits **MUST** be equal.

## 2.13 Case Study: A Fault-Tolerant Training Job That Survives Node Failures

Recall Module 2's Ray Train case study. A Kubernetes Job's `backoffLimit` lets the whole job retry if a Pod fails, but a naive retry restarts training **FROM SCRATCH**. Combining this with Module 1's MLflow checkpointing — written to a `PersistentVolume` or external object storage, never to a Pod's own ephemeral disk — lets a retried Pod resume from the last checkpoint instead of starting over.

## 2.14 Case Study: Cost-Optimization of GPU Clusters Using Spot Instances

A dedicated spot-instance node pool is tainted so only workloads that explicitly tolerate interruption are scheduled there, combined with Pod priority classes so critical workloads preempt lower-priority ones if a spot node disappears unexpectedly. Today's hyperparameter search Job is a natural fit for spot instances: each pod is independent, cheap, and quick to retry.

## Part B: Scaling Model Serving with KServe (≈70–90 minutes)

### 2.15 The Inference Scaling Problem

An always-on Deployment running a model, sized for peak traffic, wastes compute during every quiet period — nights, weekends, low-traffic hours. Today's Part B answer is a Deployment whose replica count is adjusted automatically by a HorizontalPodAutoscaler (HPA) based on real request load, load-tested and observed live across the same 3-node cluster from Part A.

### 2.16 A Tiered Hands-On Exercise

This exercise is deliberately tiered:

1. Tier 1 (required, everyone completes this): a plain Kubernetes Deployment + Service + HorizontalPodAutoscaler. This is exactly what KServe generates and manages under the hood — writing it by hand first is what makes Tier 2 feel like automation, not magic. Guaranteed to work with nothing beyond kubectl and the existing cluster.
2. Tier 2 (optional extension): install real KServe in RawDeployment mode and deploy an InferenceService wrapping the SAME container. RawDeployment avoids the Istio/Knative dependency chain that makes full KServe installs fragile on a laptop-scale cluster within a single class session; the tradeoff is that RawDeployment does not support true scale-to-zero (covered conceptually).

### 2.17 The Workload: Serving the Model from Part A

The RandomForestClassifier is trained with the best hyperparameters a student would have found in Part A's grid search, exported with joblib alongside its feature column order (JSON/dict key order is not guaranteed, so the predictor needs this explicitly), and served through a small FastAPI application implementing KServe's own V1 inference protocol (POST /v1/models/{name}:predict) — chosen specifically so Tier 1 and Tier 2 stay wire-compatible with zero client-side changes.

```
@app.post("/v1/models/{model_name}:predict")
def predict(model_name: str, request: PredictRequest):
    predictions = _bundle["model"].predict(request.instances).tolist()
    return {
        "predictions": predictions,
        "served_by_pod": POD_NAME,    # beyond the minimal V1 schema, deliberately --
        "served_by_node": NODE_NAME, # this is what makes today's scaling PROVABLE
    }
```

### 2.18 Tier 1: Deployment + Service + HorizontalPodAutoscaler

The predictor container (built the same minikube image build / minikube image load way as Part A) is deployed with a Deployment (CPU requests/limits of 250m/500m per replica, readiness and liveness probes against /healthz), fronted by a NodePort Service giving load\_test.py one stable address regardless of how many replicas currently exist, and scaled by an HPA targeting 50% average CPU utilization with minReplicas: 1 and maxReplicas: 6. The HPA requires the metrics-server add-on (minikube add-ons enable metrics-server) to have real CPU numbers to act on.

### 2.19 Hands-on: Load Testing and Observing Scale-Out

A concurrent load-testing script fires prediction requests using a thread pool at increasing concurrency levels (e.g. 1, 10, 40), recording latency and — critically — which pod and node served each request, via

the predictor's own response fields. Run alongside two live-watching terminals (`kubectl get hpa -w` and `kubectl get pods -o wide -w`), students directly observe: replica count rising as CPU utilization crosses the 50% target; new replicas landing on different nodes exactly as in Part A; and the load test's own `distinct_nodes` count climbing toward 3, which is the empirical proof that traffic is genuinely being served from multiple machines, not just multiple processes.

## 2.20 What KServe Automates

A real KServe `InferenceService`, annotated for `RawDeployment` mode, generates roughly the same `Deployment` + `Service` + `HPA` trio that Tier 1 built by hand — one declarative object instead of three.

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: rf-classifier
  annotations:
    serving.kserve.io/deploymentMode: RawDeployment
    serving.kserve.io/autoscalerClass: hpa
spec:
  predictor:
    minReplicas: 1
    maxReplicas: 6
    containers:
      - name: kserve-container
        image: rf-predictor:latest
```

## 2.21 storageUri vs. a Custom Container

In production, `spec.predictor.model.storageUri` would point directly at Module 1's MLflow Model Registry (`models:/rf-classifier/Production`) on real networked or object storage, reachable identically from any node. Today's exercise instead bakes the model directly into the container image — a deliberate classroom simplification that sidesteps minikube's node-local default storage provisioner, which would otherwise break multi-node access to a shared model file. Students should understand this as a genuine production-vs-classroom tradeoff, not a limitation of KServe itself.

## 2.22 Scale-to-Zero and Canary Rollouts (Conceptual)

Full KServe, with Knative Serving installed, can terminate an `InferenceService`'s LAST replica entirely when idle — true scale-to-zero, unavailable under `RawDeployment`'s plain-HPA model (whose `minReplicas` floor cannot go below 1). The first request after idling incurs a cold start, which is exactly why Lecture 1's minimal, multi-stage image sizes matter operationally. Canary rollouts (`canaryTrafficPercent: 10`) work identically under either deployment mode, and connect directly back to Module 1's Model Registry `stages/aliases`.

## 2.23 Case Study: Scaling an Image Classification Service During Peak Traffic

A campus photo-classification app sees near-zero traffic overnight and a sharp spike during a single lab session. Under `RawDeployment`, the service idles at `minReplicas: 1` overnight (full Knative mode could reach zero); as the session begins, concurrency rises sharply — exactly what `load_test.py` simulated — CPU utilization crosses the HPA's target, and new replicas are scheduled across all three nodes; after the session, the HPA scales back down toward its minimum over its configured stabilization window.

## References for This Module

- Docker official documentation: [docs.docker.com](https://docs.docker.com)
- NVIDIA Container Toolkit documentation: [docs.nvidia.com/datacenter/cloud-native/container-toolkit](https://docs.nvidia.com/datacenter/cloud-native/container-toolkit)
- Kubernetes official documentation: [kubernetes.io/docs](https://kubernetes.io/docs)
- Kubernetes Indexed Jobs: [kubernetes.io/docs/tasks/job/indexed-parallel-processing-static](https://kubernetes.io/docs/tasks/job/indexed-parallel-processing-static)
- Kubernetes Horizontal Pod Autoscaler: [kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale](https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale)
- minikube official documentation: [minikube.sigs.k8s.io/docs](https://minikube.sigs.k8s.io/docs)
- KServe official documentation (including RawDeployment mode): [kserve.github.io/website](https://kserve.github.io/website)